

CVM++

A Stack-Based Virtual Machine in C++17

Technical Design & Implementation Report

Full pipeline: Lexer → Parser → Compiler → VM

Language: C++17 Build: CMake 3.10+ / g++ Platform: Cross-platform

May 18, 2026

Inspired by *Crafting Interpreters* by Robert Nystrom

Contents

1	Project Overview	2
1.1	Goals and Scope	2
1.2	Supported Language Features	2
2	System Architecture	3
2.1	File Structure	3
3	Stage 1: Lexer	3
3.1	Responsibility	3
3.2	Token Types	3
3.3	Implementation Details	4
4	Stage 2: Parser	4
4.1	Responsibility	4
4.2	Grammar	4
4.3	AST Node Hierarchy	5
4.4	Error Reporting	5
5	Stage 3: Compiler	5
5.1	Responsibility	5
5.2	The Chunk Structure	5
5.3	Variable Management	6
5.4	Control Flow and Backpatching	6
6	Stage 4: Virtual Machine	6
6.1	Architecture	6
6.2	Value Representation	6
6.3	Instruction Set	7
6.4	Dispatch Loop	7
7	Entry Point and REPL	7
8	Example Programs	8
8.1	Arithmetic	8
8.2	Fibonacci Sequence	8
8.3	FizzBuzz	8
9	Build Instructions	9
9.1	Prerequisites	9
9.2	With CMake	9
9.3	Direct g++ (Windows PowerShell caveat)	9
10	Design Decisions and Limitations	10
10.1	Design Decisions	10
10.2	Known Limitations	10
11	Possible Extensions	10
12	Conclusion	11

1 Project Overview

CVM++ is a complete, from-scratch implementation of a miniature programming language and its execution engine, written in C++17. It demonstrates the full compilation pipeline that real-world language runtimes follow: source text is first broken into tokens by a **Lexer**, those tokens are structured into an **Abstract Syntax Tree (AST)** by a **Parser**, the tree is translated into flat **bytecode** by a **Compiler**, and finally that bytecode is executed by a **stack-based Virtual Machine**.

The project serves as both a learning vehicle for compiler and VM internals, and a self-contained, buildable artifact with three example programs. The codebase is approximately 900 lines of well-commented C++17 spread across nine source files.

1.1 Goals and Scope

The design priorities of CVM++ are:

- **Educational clarity** — every stage of the pipeline is isolated into its own class with a single, well-defined responsibility.
- **Correctness over performance** — the VM uses a simple sequential dispatch loop rather than computed gotos or JIT compilation.
- **Minimal dependencies** — the entire project uses only the C++17 standard library; no third-party frameworks are required.
- **Extensibility** — the architecture is intentionally layered so that new language features (strings, functions, closures) can be added without restructuring existing components.

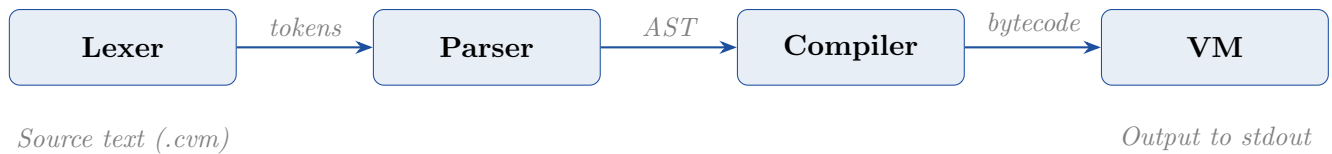
1.2 Supported Language Features

The CVM++ language supports the following constructs:

Feature	Syntax / Example
Integer literals	0, 42, -1
Boolean literals	true, false
Variable declaration	let x = 10;
Variable assignment	x = x + 1;
Arithmetic operators	+, -, *, /
Comparison operators	==, <
Print statement	print x;
Input statement	let n = input;
Conditional	if (cond) { } else { }
Loop	while (cond) { }
Single-line comment	// comment text

2 System Architecture

The compilation pipeline follows four sequential, stateless stages. Each stage consumes the output of the previous one and produces a new data structure.



2.1 File Structure

File	Responsibility
src/token.h	TokenType enum and Token struct
src/lexer.h/.cpp	Lexer: source text $\rightarrow$ <code>vector<Token></code>
src/ast.h	Full AST node hierarchy (Expr/Stmt subclasses)
src/parser.h/.cpp	Recursive-descent parser: tokens $\rightarrow$ AST
src/compiler.h/.cpp	AST walker: emits bytecode into a <code>Chunk</code>
src/opcode.h	OpCode enum shared between compiler and VM
src/vm.h/.cpp	Stack-based bytecode interpreter
src/main.cpp	Entry point: file runner and interactive REPL
CMakeLists.txt	CMake build configuration

3 Stage 1: Lexer

3.1 Responsibility

The `Lexer` class (`lexer.h/.cpp`) takes the raw source string and produces a flat `vector<Token>`. A `Token` is a struct holding three fields: a `TokenType` enum value, the raw text (`value`), and the source line number for error reporting.

3.2 Token Types

Category	Token types
Literals	INT_LIT, TRUE_KW, FALSE_KW
Keywords	LET, IF, ELSE, WHILE, PRINT, INPUT
Identifiers	IDENT
Operators	PLUS, MINUS, STAR, SLASH, ASSIGN, EQ_EQ, LESS
Delimiters	LPAREN, RPAREN, LBRACE, RBRACE, SEMICOLON
Control	EOF_T

3.3 Implementation Details

The lexer maintains a position cursor `pos` and a `line` counter. It iterates character-by-character through the source string:

- **Whitespace and comments** are consumed by `skipWhitespaceAndComments()`, which handles spaces, tabs, carriage returns, newlines, and `//` line comments.
- **Numbers** are read by `readNumber()`, which advances while `isdigit()` is true and constructs an `INT_LIT` token.
- **Identifiers and keywords** are read by `readIdentOrKeyword()`, which scans alphanumeric or underscore characters and performs a lookup against a static `unordered_map` keyword table.
- **Single- and two-character tokens** (`=` vs `==`) are handled by a `switch` statement with one lookahead character.
- **Unknown characters** raise a `std::runtime_error` with the offending line number.

4 Stage 2: Parser ---

4.1 Responsibility

The `Parser` class (`parser.h/.cpp`) implements a **recursive-descent parser** that converts the flat token list into a typed Abstract Syntax Tree rooted at a `Program` node.

4.2 Grammar

The grammar implemented is:

```

program    -> stmt* EOF
stmt       -> letStmt | assignStmt | ifStmt | whileStmt | printStmt
letStmt    -> "let" IDENT "=" expr ";"
assignStmt -> IDENT "=" expr ";"
ifStmt     -> "if" "(" expr ")" block ("else" block)?
whileStmt  -> "while" "(" expr ")" block
printStmt  -> "print" expr ";"
block      -> "{" stmt* "}"
expr       -> comparison
comparison -> addition ( ("==" | "<") addition ) *
addition   -> multiply ( ("+" | "-") multiply ) *
multiply   -> primary ( ("*" | "/" ) primary ) *
primary    -> INT_LIT | "true" | "false" | IDENT | "input" | "(" expr ")"

```

Operator precedence is baked into the grammar hierarchy: multiplication and division bind tighter than addition and subtraction, which bind tighter than comparison operators. This is standard practice in recursive-descent parsers.

4.3 AST Node Hierarchy

The AST is defined entirely in `ast.h` using virtual base classes and `unique_ptr` ownership:

Node Type	Represents
<i>Expressions (subclass of Expr)</i>	
IntLitExpr	Integer constant, e.g. 42
BoolLitExpr	Boolean constant <code>true</code> / <code>false</code>
VarExpr	Variable read, e.g. <code>x</code>
BinaryExpr	Binary operation with operator char and two child Exprs
InputExpr	Read integer from stdin
<i>Statements (subclass of Stmt)</i>	
LetStmt	Variable declaration with initializer expression
AssignStmt	Variable re-assignment
PrintStmt	Print an expression to stdout
IfStmt	Conditional with optional else block
WhileStmt	Loop with condition and body
Program	Root node, holds a StmtList

4.4 Error Reporting

Every `expect()` call on a mismatched token throws a `std::runtime_error` that includes the line number and the offending token text, e.g.: *“Parse error at line 5: Expected ‘;’ after let statement (got ‘while’)”*.

5 Stage 3: Compiler

5.1 Responsibility

The `Compiler` class (`compiler.h/.cpp`) performs a single-pass walk of the AST and emits a flat byte array (a **Chunk**) that encodes the program as bytecode instructions for the VM.

5.2 The Chunk Structure

A **Chunk** is a thin wrapper around `vector<uint8_t>` with three helper methods:

- `emit(byte)` — append a single byte.
- `emitInt32(v)` — append a 32-bit integer in big-endian order (four bytes), used for `PUSH_INT`.
- `emitJump(op)` — emit a jump opcode followed by a two-byte placeholder `0xFFFF`.
- `patchJump(pos, target)` — overwrite the placeholder at `pos` with the real target address (backpatching).

5.3 Variable Management

Variables are stored in a flat array of 256 slots inside the VM. The compiler maintains an `unordered_map<string, uint8_t>` that maps variable names to slot indices. Declaring a variable with `let` allocates the next available slot; re-declaring a variable reuses the existing slot (idiomatic for loop variables like `tmp` in the Fibonacci example).

The maximum number of variables is 255, enforced at compile time.

5.4 Control Flow and Backpatching

Because the target address of a jump is not known when the jump instruction is first emitted (the compiler hasn't yet generated the code it jumps over), a two-phase approach is used:

1. **Emit a placeholder:** `emitJump(JUMP_IF_FALSE)` writes the opcode and two `0xFF` bytes. The position of those bytes is saved.
2. **Compile the body** of the branch or loop.
3. **Patch the placeholder:** `patchJump(saved_pos, chunk.here())` overwrites the two bytes with the now-known target address.

For `while` loops, the back-edge jump is emitted *after* the body, pointing back to the saved `loop_start` position recorded before compiling the condition.

6 Stage 4: Virtual Machine

6.1 Architecture

The VM (`vm.h/.cpp`) is a **stack-based interpreter**. All operations work on an implicit operand stack (`vector<Value>`). There are no registers. The instruction pointer `ip` is a `size_t` index into the bytecode array.

6.2 Value Representation

Runtime values are represented by the `Value` struct: a tagged union holding either a 32-bit integer or a boolean.

Listing 1: Value struct from `vm.h`

```
1 enum class ValType { INT, BOOL };
2
3 struct Value {
4     ValType type = ValType::INT;
5     union { int32_t i = 0; bool b; };
6
7     static Value fromInt (int32_t v) { ... }
8     static Value fromBool (bool v) { ... }
9
10    bool truthy() const {
11        return (type == ValType::BOOL) ? b : (i != 0);
```

```

12     }
13 };

```

Truthiness is defined as: boolean `true`, or any non-zero integer.

6.3 Instruction Set

The complete instruction set, as defined in `opcode.h`:

Opcode	Operand bytes	Effect
PUSH_INT	[b3] [b2] [b1] [b0]	Push 32-bit int (big-endian)
PUSH_TRUE	---	Push boolean <code>true</code>
PUSH_FALSE	---	Push boolean <code>false</code>
LOAD	[idx]	Push <code>vars[idx]</code> onto stack
STORE	[idx]	Pop $\rightarrow$ <code>vars[idx]</code>
ADD	---	Pop b, a ; push $a + b$
SUB	---	Pop b, a ; push $a - b$
MUL	---	Pop b, a ; push $a \times b$
DIV	---	Pop b, a ; push a/b (integer)
EQ	---	Pop b, a ; push $a == b$
LESS	---	Pop b, a ; push $a < b$
PRINT	---	Pop and print to stdout
INPUT	---	Read int from stdin, push
JUMP	[hi] [lo]	$ip \leftarrow$ address
JUMP_IF_FALSE	[hi] [lo]	Pop; if falsy $\rightarrow ip \leftarrow$ address
POP	---	Discard top of stack
HALT	---	Stop execution

6.4 Dispatch Loop

The VM runs a `while(true)` loop that reads one opcode byte, then switches on it. Each case reads any operand bytes, manipulates the stack, and either continues to the next iteration or `returns` on `HALT`. Division by zero and stack underflow raise `std::runtime_error`.

7 Entry Point and REPL

`main.cpp` supports two modes of operation:

- **File mode** (`./cvm script.cvm`): reads the entire file into a string and passes it through the four-stage pipeline via `runSource()`.
- **REPL mode** (`./cvm`): displays a welcome banner and repeatedly prompts with `>>`, running each line through the full pipeline independently. The loop exits on `exit`, `quit`, or EOF.

All errors caught by `std::exception` are printed to `stderr` in red using ANSI escape codes, and the REPL continues rather than terminating.

8 Example Programs

Three example programs are included in the `examples/` directory.

8.1 Arithmetic

Demonstrates basic variable declarations and all four arithmetic operators on integer values. The program computes sum, difference, product, and integer quotient of `a = 15` and `b = 4`.

Listing 2: arithmetic.cvm

```

1 let a = 15;
2 let b = 4;
3 let sum  = a + b;
4 let diff = a - b;
5 let prod = a * b;
6 let quot = a / b;
7
8 print sum;    // 19
9 print diff;   // 11
10 print prod;  // 60
11 print quot;  // 3

```

8.2 Fibonacci Sequence

Prints the first 10 terms of the Fibonacci sequence using a `while` loop and a temporary variable to perform the swap.

Listing 3: fibonacci.cvm

```

1 let a = 0;
2 let b = 1;
3 let i = 0;
4 while (i < 10) {
5     print a;
6     let tmp = a + b;
7     a = b;
8     b = tmp;
9     i = i + 1;
10 }

```

Output: 0 1 1 2 3 5 8 13 21 34

8.3 FizzBuzz

Prints FizzBuzz for integers 1 through 20. Because CVM++ has no modulo operator, divisibility is checked using integer division and multiplication:

$$n \bmod d = 0 \iff \left\lfloor \frac{n}{d} \right\rfloor \times d = n$$

Since the language also lacks string support, numeric markers are used in place of text: 0 for FizzBuzz, 1 for Fizz, 2 for Buzz.

Listing 4: fizzbuzz.cvm (condensed)

```

1 let i = 1;
2 while (i < 21) {
3     let d15 = i / 15;
4     let d3  = i / 3;
5     let d5  = i / 5;
6     if ((d15 * 15) == i) {
7         print 0;          // FizzBuzz
8     } else {
9         if ((d3 * 3) == i) {
10            print 1;       // Fizz
11        } else {
12            if ((d5 * 5) == i) {
13                print 2; // Buzz
14            } else {
15                print i;
16            }
17        }
18    }
19    i = i + 1;
20 }

```

9 Build Instructions

9.1 Prerequisites

- C++17-capable compiler: GCC 7+, Clang 5+, or MSVC 2017+.
- CMake 3.10+ (recommended), or any direct g++ invocation.

9.2 With CMake

```

mkdir build && cd build
cmake ..
cmake --build .
./cvm ../examples/fibonacci.cvm

```

9.3 Direct g++ (Windows PowerShell caveat)

On Linux/macOS, the backslash line continuation works:

```

g++ -std=c++17 -Isrc src/main.cpp src/lexer.cpp src/parser.cpp \
    src/compiler.cpp src/vm.cpp -o cvm

```

On **Windows PowerShell**, the backslash must be replaced with a backtick, or the command written on a single line:

```

g++ -std=c++17 -Isrc src/main.cpp src/lexer.cpp src/parser.cpp src/compil

```

10 Design Decisions and Limitations

10.1 Design Decisions

- **Single-pass compilation:** the compiler walks the AST once and emits bytecode directly, using backpatching to handle forward jumps. This is simple and sufficient for a language without forward-declared functions.
- **dynamic_cast for AST dispatch:** the compiler uses `dynamic_cast` on the polymorphic node hierarchy rather than the Visitor pattern. This is slightly less efficient but keeps the code readable and avoids boilerplate.
- **Flat variable slots:** all variables are stored in a 256-element array indexed by a compile-time `uint8_t`. There is no scoping or shadowing — all variable names are globally unique within a program.
- **Big-endian integer encoding:** `PUSH_INT` encodes 32-bit integers in big-endian order for straightforward human inspection of raw bytecode.

10.2 Known Limitations

Limitation	Impact
No string type	Output must use numeric proxies (see FizzBuzz)
No modulo operator	Divisibility requires the $\lfloor n/d \rfloor \times d = n$ trick
No negation operator	Negative literals not parsed; use <code>0 - x</code>
255-variable cap	Enforced at compile time; sufficient for small programs
No functions	No user-defined subroutines or recursion
No scoping	All variables are global within a program
16-bit jump targets	Programs limited to 65535 bytes of bytecode

11 Possible Extensions

The architecture cleanly supports the following additions without restructuring any existing component:

- **String type:** add a `ValType::STRING` variant to `Value`, a `PUSH_STR` opcode, and a string pool in the `Chunk`.
- **Modulo operator:** add a `%` token, a `MOD` opcode, and the corresponding VM case.
- **Unary negation:** add a `NEG` opcode and a unary prefix parse rule.
- **Functions and closures:** requires a call stack (frames), a `CALL/RETURN` opcode pair, and upvalue support in the VM — the primary direction for a follow-on project. See *Crafting Interpreters* Chapters 24–28.
- **Disassembler:** a debug utility that pretty-prints bytecode offsets, opcodes, and operands from a compiled `Chunk`.

- **Greater-than / and / or operators:** straightforward additions to the lexer, parser grammar, compiler, and opcode table.

12 Conclusion

CVM++ is a clean, well-structured implementation of a complete language runtime in under 1000 lines of C++¹⁷. It faithfully reproduces the architecture of production language implementations — lexical analysis, recursive-descent parsing, AST compilation, and stack-based bytecode execution — at a scale that is comprehensible in a single reading session.

The project demonstrates key systems programming concepts: tagged union runtime values, backpatching for forward jumps, operator-precedence encoding in grammar rules, and the separation of concerns between compilation and execution. It serves as an ideal foundation for exploring more advanced topics such as closures, garbage collection, and JIT compilation.

Reference: “Crafting Interpreters” by Robert Nystrom — highly recommended for extending this project with strings, functions, and closures.